

Master-Track 4 — Rollenquelle und digitale Signaturen

Datum: 28.08.2026

Ausgangsstand: dcfb61e (Bonussystem gehärtet), 7468 vitest, 2476 node:test

Live-Nachweis: scripts/verify-signaturen-live.mjs — 11/11 grün

Migration nötig: nein

Kurzfassung

Zwei Blöcke, beide gegen die Produktionsdatenbank nachgeprüft.

Block A schließt eine Spaltung, die das ganze Projekt betrifft: es gab zwei autoritative Rollenquellen und zwei gegenläufige Lesarten davon. Der Torwächter (proxy.ts) bevorzugte `app_metadata.role` und fragte `profiles` gar nicht erst ab; die dreizehn Fach-Guards lasen ausschließlich `profiles.role`. Praktisch heißt das: ****eine Herabstufung in der Datenbank wirkte im Torwächter nicht.****

Block B härtet das Signaturmodul. Der zentrale Befund: der Nachweis, um den es in dem Modul geht, konnte nie entstehen — jede Unterschrift eines Nicht-Administrators scheiterte am Audit-Eintrag, nachdem sie schon geschrieben war. Alle vier Tabellen sind live leer, was dazu passt.

Block A — Zwei Rollenquellen, zwei Antworten

Befund A-1 (P0): Rechteentzug in der Datenbank blieb im Torwächter wirkungslos

`lib/auth/rollen.ts` nennt seit dem Rollenkonzept zwei Quellen, die beide nicht vom Nutzer selbst beschreibbar sind:

- `app_metadata.role` — nur über die GoTrue-Admin-API setzbar
- `profiles.role` — durch den Trigger `prevent_role_escalation` geschützt

Gelesen wurden sie an zwei Stellen unterschiedlich:

Schicht	Regel bis 28.08.2026
<code>proxy.ts</code> (Bereichssperre, im Code als „verbindlich“ bezeichnet)	<code>app_metadata.role</code> gewinnt; <code>profiles</code> wurde gar nicht abgefragt, wenn <code>app_metadata</code> gesetzt war
<code>lib/auth/guard.ts</code> (<code>requireBerechtigung</code> , <code>requireAdministration</code>)	<code>`appRole \</code>
<code>app/admin/layout.tsx</code> , <code>app/auth/login/page.tsx</code>	dieselbe Reihenfolge
13 Fach-Guards (<code>lib/**/api-auth.ts</code>)	ausschließlich <code>profiles.role</code>

`app_metadata.role` wird an genau einer Stelle geschrieben: `/api/admin/manage-role`. Jede Herabstufung, die direkt in der Datenbank passiert — der dokumentierte Weg für `superadmin` und der einzige Weg für eine Korrektur außerhalb dieser Route — hinterlässt den alten, höheren Wert im Token. Danach:

- proxy.ts ließ die Person weiter in /admin und /mis,
- requireBerechtigung gewährte weiter die alten Berechtigungen,
- die Navigation zeigte weiter die alten Bereiche,
- die 13 Fach-Guards wiesen ab.

Ein Entzug, der nur zur Hälfte wirkt, ist kein Entzug. Zweiter, kleinerer Nebenfund derselben Stelle: ein Token mit app_metadata.role ohne zugehörigen profiles-Datensatz bekam über holeRolle() volle Rechte — profile?.role war nur der Rückfall, nicht die Bedingung.

Regel ab jetzt

lib/auth/rollen.ts, Block „Zwei autoritative Quellen“:

1. profiles ist bindend. Fehlt der Datensatz oder trägt er keine bzw. eine unbekannte Rolle, gibt es keine Berechtigung — gleich, was im Token steht. Das entspricht dem bisherigen Verhalten der Fach-Guards (if (!profile) → 403).
2. app_metadata wirkt nur einschränkend. Ist es gesetzt, gilt die Schnittmenge beider Berechtigungslisten.
3. Ist app_metadata nicht gesetzt, entscheidet profiles allein — der unveränderte Bestandsfall.

Die Regel kann per Konstruktion nur Rechte nehmen, nie geben; ein Regressionstest prüft das über alle 9 × 9 Rollenpaare. Eine Rechtevergabe verlangt weiterhin, dass beide Quellen zustimmen — /api/admin/manage-role schreibt genau deshalb beide.

> Betriebliche Folge, die festgehalten werden muss: Wer eine Rolle
 > hochstuft, muss das über /api/admin/manage-role tun oder beide Quellen
 > schreiben. Eine reine UPDATE profiles SET role = ...-Hochstufung wirkt bei
 > einem Konto, dessen app_metadata.role bereits einen niedrigeren Wert
 > trägt, nicht mehr. Das ist die fail-closed-Richtung und beabsichtigt; unter
 > der alten Regel widersprachen sich in genau diesem Fall Torwächter und
 > Fach-Guards.

Umsetzung

- lib/auth/rollen.ts: wirksameBerechtigungen, wirksamDarf, wirksamDarfEines, wirksamDarfAlle, wirksamIstAdministration, wirksamIstVerwaltungsrolle, wirksameRolle.
- Neu: lib/auth/rollen-quelle.ts — holeRollenQuellen(supabase) und quellenDuerfen(). Eine Ermittlung statt dreizehn Kopien desselben Blocks.
- Umgestellt: proxy.ts, lib/auth/guard.ts, app/admin/layout.tsx, app/auth/login/page.tsx und die zwölf Fach-Guards (abrechnung, akten, angehorige, expansion, kim, medikamente, ops, personal, pflege, signaturen, uebergabe, wunden). lib/coach/api-auth.ts liest keine Rolle und blieb unverändert.
- requireOpsAdmin — die im Auftrag genannten 48 Routen — hängt an lib/ops/api-auth.ts und ist damit mit umgestellt.

Kosten: Bei Konten mit gesetztem app_metadata.role kommt in proxy.ts eine zusätzliche, indizierte profiles-Abfrage pro Anfrage dazu. Bei allen übrigen Konten lief sie ohnehin schon — dort ändert sich nichts.

Was hier NICHT umgestellt wurde

Rund hundert Routen und Server Actions lesen profiles.role inline (app/api/billing/dta/, app/api/admin/clients/, die */actions.ts der Portale). Sie lesen damit bereits die bindende Quelle und sind von der

Spaltung nicht betroffen; die zusätzliche Einschränkung aus `app_metadata` wenden sie nicht an. Das ist ein Restposten, kein offener Fail-open — und bewusst ein eigener Track, weil er hundert Aufrufstellen berührt.

Block B — Digitale Signaturen

Alle Befunde sind am 28.08.2026 live gegen Produktion nachgeprüft (`scripts/verify-signatures-live.mjs`).

Befund B-1 (P0): Der Nachweis konnte nie entstehen

`signatur_audit_log` trägt live genau eine permissive Policy: `admin_sig_audit_all` mit `is_admin()`, und `is_admin()` ist live `admin|superadmin` (aus `pg_proc.prosrc` gelesen). `protokolliereSignaturAudit()` lief aber mit dem RLS-Client des Aufrufers.

Ein Signatar ist per Definition selten die Administration. Er darf seine eigene Zeile in `signatures` schreiben (Policy `signatar_eigene_update`, live vorhanden), den Nachweis darüber aber nicht. Der Ablauf war deshalb:

1. Unterschrift wird geschrieben — erfolgreich.
2. `INSERT` in `signatur_audit_log` scheitert an RLS (42501).
3. Die Funktion wirft, die Route antwortet HTTP 500.
4. Der Signatar sieht einen Fehler. Die Unterschrift steht trotzdem.
5. Zweiter Versuch: „Signatur hat Status ‚signiert‘ — kann nicht signiert werden.“

Der Nachweis — das, wofür das Modul da ist (§ 630f BGB, Art. 30/32 DSGVO) — entstand nie. Live sind alle vier Tabellen leer, was zu „der Weg hat nie funktioniert“ passt. Dieselbe Klasse wie das Angehörigenportal (48d6f3b).

Abhilfe: Der Nachweis läuft über den Dienstschlüssel und `fail-closed`. Der Statuswechsel wird per `Compare-and-Swap` beansprucht, dann der Nachweis geschrieben, und scheitert er, wird der Wechsel zurückgenommen und die Route antwortet 503 mit Klartext. Die Rücknahme muss über den Dienstschlüssel laufen: `signatar_eigene_update` trägt `status = 'offen'` im `USING`, der Signatar kommt an seine eigene Zeile nach dem Wechsel also nicht mehr heran.

Befund B-2 (P0): `verifiziereSignatur` prüfte den Dokumentinhalt nicht

Die Funktion lud `dokument_inhalt_snapshot` und benutzte ihn nicht. Geprüft wurde allein, ob der gespeicherte Signatur-Hash zu (Dokument-Hash, Signatar, Zeitstempel) passt. Wer den Dokumentinhalt nachträglich ändert und den Hash stehen lässt, bekam `gueltig: true` — also genau in dem Fall, für den die Prüfung existiert.

Abhilfe: Zwei Prüfungen statt einer. Das Ergebnis nennt beide getrennt (`signaturHashStimmt`, `dokumentUnveraendert`). Ohne Schnappschuss ist der Inhalt nicht prüfbar; das Ergebnis sagt das ausdrücklich (`dokumentUnveraendert: null` plus Hinweistext) und behauptet keine Unversehrtheit, die es nicht belegen kann.

Befund B-3 (P1): `x-forwarded-for` roh in eine `inet`-Spalte

`signatures.ip_adresse` ist live vom Typ `inet`. Die Route schrieb den rohen Header hinein. Hinter einer Proxy-Kette steht dort `"203.0.113.7, 198.51.100.4"` — Postgres weist das mit 22P02 ab (live gegengeprüft), das `UPDATE` scheitert, und die Unterschrift geht komplett

verloren. Jede andere Stelle im Repo nimmt den ersten Eintrag (lib/audit-log.ts, lib/rate-limit.ts, app/api/track/route.ts, ...) — diese war die einzige, die es nicht tat.

Abhilfe: erstelpAdresse() nimmt den ersten Eintrag, schneidet einen angehängten Port ab, prüft die Form und liefert sonst null. Die IP ist Begleitinformation, nicht der Beweis — sie darf die Unterschrift nicht zum Scheitern bringen. user_agent wird auf 512 Zeichen gekürzt.

Befund B-4 (P1): pdl/qm/buchhaltung bekamen stille leere Listen

signatur_dokumente hat live keine Policy für diese Rollen, der Guard ließ sie über einsatz.lesen aber herein. Ergebnis: leere Liste ****ohne Fehlermeldung**** beim Lesen, 42501 als „Interner Serverfehler“ beim Schreiben. Dieselbe stille Falschauskunft wie in d707cda und 48d6f3b.

Abhilfe auf derselben Linie wie dort: Dienstschlüssel plus Fence im Code (organization_id und Erlaubnisliste der Dokumentarten in jeder Abfrage). Bewusst keine neue RLS-Policy: der Dienstschlüssel umgeht sie ohnehin, und eine Lese-Policy auf signaturen gäbe über PostgREST mehr her, als die Schnittstelle zeigen soll.

Befund B-5 (P1): Der Dokumenttyp bestimmte die Berechtigung nicht

Alle Routen verlangten pauschal einsatz.lesen/einsatz.schreiben. Die Tabelle führt aber sechs Arten quer über drei Fachbereiche — pflegebericht sind Gesundheitsdaten. Die Buchhaltung, die ausdrücklich keine Gesundheitsdaten sehen soll, hat einsatz.lesen und bekam damit Pflegeberichte in ihre Liste.

Abhilfe: lib/signaturen/berechtigung.ts ordnet jede Art einem Fachbereich zu:

Dokumentart	Bereich
leistungsnachweis, protokoll	einsatz.*
pflegebericht	pflege.*
vertrag, einwilligung	stammdaten.*
sonstiges	nicht zugeordnet — nur Administration

sonstiges hat per Definition keinen erklärten Inhalt. Einen Katalogtyp ohne Aussage einem Fachbereich zuzuschlagen hieße raten; geraten wird hier nicht (Grundsatz 1: verweigern ist der Normalfall). Wer gar keine Art sehen darf, bekommt 403 und keine leere Liste.

Befund B-6 (P1): Statuswechsel ohne Compare-and-Swap

leisteSignatur und lehneSignaturAb prüften den Status und schrieben dann ohne Statusbedingung im UPDATE. Für Signaturen fängt die DB-Policy das ab (status = 'offen' im USING), für die Administration greift is_admin() FOR ALL ohne Statusbedingung — dort konnte eine abgelehnte Signatur im Rennen zu einer signierten werden. Jetzt CAS auf status = 'offen' plus signatar_id plus organization_id; greift er ins Leere, ist die Antwort 409 mit Klartext statt eines stillen Überschreibens.

Befund B-7 (P2): Sammelposten

- Rohe Fehlermeldungen. Drei Routen umgingen safeApiError und gaben

err.message mit HTTP 500 aus — inklusive Postgres-Text. Jetzt durchgängig UserFacingError + safeApiError; die Validierer in lib/signaturen/types.ts nennen Feld und zulässige Werte, statt vom Sanitizer zu „Interner Serverfehler“ verkürzt zu werden (gleiche Stelle wie in lib/personal, faa0972).

- signatar_id ungeprüft. Der Fremdschlüssel zeigt auf auth.users; ein Tippfehler ergab 23503 → 500. Jetzt Prüfung gegen profiles, 404 mit Klartext.
- Dokument-Hash aus dem Body. Liegt ein Inhalts-Schnappschuss vor, ist der Hash eine Ableitung davon und wird serverseitig berechnet; ein mitgeschickter, abweichender Wert ist ein Fehler (400) und keine zweite Meinung. Ohne Schnappschuss bleibt der Hash Pflichtfeld.
- verifizieren war für jedes angemeldete Konto offen. requireSigUser hat keine Rollenprüfung — die Aktion nennt aber Signatar, Zeitpunkt und Methode. Jetzt: der Signatar selbst oder eine Rolle, die die Dokumentart lesen darf.
- Unterschriftsbild ohne Größengrenze. signatur_daten ist unbegrenztes text; jetzt gegen MAX_BILD_BYTES (5 MB) geprüft, 413.
- sendeQesAnfrage warf einen nackten Error; jetzt UserFacingError 501.

Live-Nachweis

node scripts/verify-signaturen-live.mjs — 11/11 grün:

Prüfung	Ergebnis
signatur_dokumente, signaturen, signatur_audit_log, qes_hooks	live erreichbar, je 0 Zeilen
signatur_audit_log: permissive Policies	genau 1, auf is_admin() — ein Signatar kann per RLS nicht protokollieren
signaturen.signatar_eigene_update	vorhanden — der Signatar darf seine Zeile schreiben, den Nachweis dazu nicht
signatur_dokumente: Policy für pdl/qm/buchhaltung	keine
is_admin()	role = ANY (ARRAY['admin','superadmin']) AND deleted_at IS NULL
signaturen.ip_adresse	inet
inet gegen '203.0.113.7, 198.51.100.4'	22P02, abgewiesen
inet gegen '203.0.113.7'	angenommen

Nicht verifizierbar in dieser Sitzung: die tatsächliche Verteilung von app_metadata.role in Produktion. /auth/v1/admin/users antwortet mit HTTP 500 („Database error finding users“), und auth.users ist über das SQL-Leseorakel nicht lesbar („permission denied for table users“). Befund A-1 ist deshalb als Code-Eigenschaft belegt, nicht als beobachteter Datenzustand — die Regel gilt unabhängig davon.

Live-Bestand zur Einordnung: 6 Organisationen, 63 Profile (34 kunde / 20 engel / 5 fahrer / 3 superadmin / 1 admin), 4 Klienten, 2 Betreuungskräfte, 3 Leistungsnachweise, 3 Rechnungen.

Prüfläufe

Lauf	Vorher	Nachher
vitest run	7468 bestanden	7532 bestanden, 0 rot (332 Dateien, 38 uebersprungen)
npm run test:unit (node:test)	2476	2476 bestanden, 0 rot (unveraendert)
tsc --noEmit	0 Fehler	0 Fehler
npm run lint:forbidden	0	0 (24 761 Dateien, Vollscan)

Neue Tests: 64 (7468 → 7532). Die Gegenproben führen die alte Regel noch einmal aus und zeigen, dass sie das Gegenteil ergab:

- alte Rollenregel ließ einen in der Datenbank herabgestuften Admin durch — als reine Funktion (`__tests__/security/rollenquelle-wirksam.test.ts`) und am echten Torwächter (`app_metadata=admin, profiles=kunde → Redirect nach /kunde/home, __tests__/security/p0-1-admin-auth.test.ts`),
- alte Rollenregel gab einem Token ohne Profil-Zeile volle Rechte; jetzt Redirect zum Login,
- Torwächter und Fach-Guards gaben demselben Nutzer zwei verschiedene Antworten — jetzt dieselbe,
- eine fehlgeschlagene profiles-Abfrage führt zum Login statt unbemerkt zu bleiben (supabase-js wirft bei PostgREST-Fehlern nicht),
- der rohe x-forwarded-for-Wert enthält das Komma, das inet ablehnt,
- ein nachträglich geänderter Dokumentinhalt bei weiterhin passendem Signatur-Hash — der Fall, den die alte Prüfung „gültig“ nannte,
- ein fehlgeschlagener Nachweis nimmt Unterschrift bzw. Ablehnung nachweislich zurück (zweites UPDATE mit status: 'offen').

Angepasste Bestandstests (die alte Regel war dort festgeschrieben):
p0-1-admin-auth, angehorigenportal-routenschutz, require-admin (Doppelgänger um maybeSingle ergänzt und um die zweite Rollenquelle), p0-pflege-mandanten-isolation und rollenkonzept-zugriffe (Scan auf die neue Form der Berechtigungsfrage), rollenquelle-und-nachweis-audit (Login-Weiterleitung liest jetzt beide Quellen).

Migration

Keine. Beide Blöcke sind Code-Änderungen. Die Signatur-Policies stehen live bereits richtig (`is_admin()`); der Fehler war, dass die Anwendung ihnen mit dem falschen Client vorauslief. Eine RLS-Policy für `pdl/qm/buchhaltung` wäre die falsche Abhilfe — der Dienstschlüssel umgeht sie ohnehin, und über PostgREST gäbe sie mehr her als die Schnittstelle zeigen soll.

Nicht geprüft (offen für Folge-Tracks)

Der Auftrag nannte mehrere Bereiche; geprüft und behandelt wurden `requireOpsAdmin/Rollenquelle` und die Dokumenten-Signierung. Nicht Teil dieses Tracks:

- CAMT/Banking. Nur die Tabellenlage erhoben (`camt_imports`, `sepa_mandates`, `sepa_batches`, `zahlungseingaenge`, `payment_allocations`,

payment_differences) — keine inhaltliche Prüfung.

- Klienten-Portal. Stichprobe: app/kunde/** arbeitet durchgängig mit Server Actions über den RLS-Client, es gibt dort keinen Dienstschlüssel-Pfad. Damit fehlt die Fehlerklasse, die im Angehörigenportal und in der Personalverwaltung gefunden wurde. Eine vollständige Prüfung der Freigabelogik steht aus.

- Automatisierungsketten 1-8. Der Orchestrator (lib/automation/index.ts) läuft fehlertolerant je Kette, der Cron (/api/cron/automatisierung, täglich 05:00 laut vercel.json) iteriert alle Organisationen. Kleiner Nebenbefund, nicht behoben: er filtert organizations nicht auf aktive Mandanten. Welche Ketten in Produktion tatsächlich Wirkung erzielen, ist ohne CRON_SECRET-Lauf nicht belegbar (siehe memory/cron-secret-gruener-lauf-kein-beweis.md).

- Dashboard-Kennzahlen. Nicht angefasst.

- Die rund hundert Inline-profiles.role-Lesungen (siehe Block A, „Was hier nicht umgestellt wurde“).